ADCS Simulink Model Reading Guide

A technical handbook for understanding the `adcs-sim` model

Prepared from the repository source of truth

April 23, 2026

Abstract

This handbook explains how to read the `adcs-sim` MATLAB/Simulink model as it exists in the repository today. It is written for a beginner who can open the model and recognize blocks and connectors, but cannot yet tell what each signal means, how the subsystems fit together, or where to look in the source tree when the diagram alone is not enough. The guide covers the architecture, inputs, outputs, GUI controls, signal flow, subsystem responsibilities, key equations, and current modeling limitations. It also distinguishes clearly between the simulator architecture that the project is designed to support and the compile-safe fallback behavior that several blocks currently use in the validated branch.

Executive Summary

The most important fact to understand before reading this model is that the generated `adcs_sim.slx` file is **not** the real source of truth. The source of truth is the MATLAB code that builds the model:

```
build_adcs_model.m  ->  builders/build_*.m  ->  adcs_sim.slx
```

For a beginner, that one fact removes most of the confusion. If a connector is visually hard to follow in Simulink, the definitive answer is usually in one of three places:

1. `build_adcs_model.m`, where top-level subsystems and cross-subsystem lines are created.
2. `reference.md`, which lists port orders and signal names in plain language.
3. The relevant builder file in `builders/`, where the subsystem interface and internal block logic are declared.

At a high level, the model is organized as a left-to-right closed-loop chain:

```
ENVIRONMENT -> SENSORS -> CONTROL -> ACTUATORS -> DYNAMICS -> workspace logs
                    ^                                    |
                    +------------- feedback -------------+
```

That loop is the conceptual backbone of the project:

- **ENVIRONMENT** provides truth-side orbital and environmental quantities such as Sun direction, Moon direction, magnetic field, position, velocity, and eclipse state.
- **SENSORS** converts truth-side states into measurements such as attitude, angular rate, body-frame Sun vector, body-frame magnetic field, and GNSS-like position and velocity.
- **CONTROL** consumes measurements and decides what the spacecraft should do.
- **ACTUATORS** convert those commands into torques or momentum-change signals.
- **DYNAMICS** integrates the rotational state and feeds true attitude and rate back into the sensor models.
- **To Workspace** blocks save the main state histories for post-processing and GUI display.

The second critical fact is that the current validated branch is **architecturally comprehensive but behaviorally mixed-fidelity**. The dataflow is real and the rigid-body attitude dynamics path is meaningful, but several blocks presently run compile-safe fallback implementations. In particular:

- Orbit propagation is currently a fixed finite LEO state fallback.
- Magnetic field is currently a constant-vector fallback.
- Eclipse is currently a constant sunlit fallback.
- Star tracker, IMU, and GNSS are currently passthrough sensor fallbacks.
- The onboard ephemeris, reference generator, mode manager, control law, reaction wheel assembly, magnetorquer assembly, and CMG are all currently simplified fallback implementations.
- The main physically active pieces in the validated branch are quaternion kinematics, quaternion normalization, rigid-body rotational dynamics, and gravity-gradient torque.

That means this handbook should be read in two layers:

- **Layer 1: architecture and interfaces.** These are already worth studying and are stable enough to understand the intended ADCS pipeline.
- **Layer 2: present numerical fidelity.** This is only partial in the current branch and must be described honestly.

Contents

Executive Summary **1**

1 What the Model Is **5**

1.1 Generated versus authored content 5

2 How to Read the Model Without Getting Lost **6**

2.1 How connectors work in this project . . . 6

3 End-to-End Simulation Workflow **7**

3.1 What the GUI actually does . . . 7

4 Top-Level Architecture and Data Flow **7**

4.1 Top-level signal flow . . . 8

4.2 What this architecture means physically . . . 8

5 Inputs to the Model **9**

5.1 Base-workspace parameter categories . . . 9

5.2 CSV ephemeris input . . . 10

5.3 GUI controls . . . 10

5.4 What the GUI does not change . . . 11

6 Outputs from the Model **12**

6.1 Primary logged outputs . . . 12

6.2 Derived metrics . . . 12

6.3 Default-run interpretation . . . 14

7 Subsystem-by-Subsystem Guide **15**

7.1 ENVIRONMENT . . . 15

7.2 SENSORS . . . 15

7.3 CONTROL . . . 16

7.4 ACTUATORS . . . 17

7.5 DYNAMICS . . . 18

8 Physics and Math Background for Hard-to-Read Parts **19**

8.1 Frames . . . 19

8.2 Quaternions . . . 20

8.3 Rigid-body rotational dynamics . . . 20

8.4 Gravity-gradient torque . . . 20

8.5 Magnetorquer torque . . . 21

8.6 Reaction-wheel momentum . . . 21

9 Detailed End-to-End Data Path **21**

9.1 Step 1: Environment truth is assembled . . . 21

9.2 Step 2: Sensors reinterpret truth as measurements . . . 21

9.3 Step 3: Control decides what to command . . . 22

9.4 Step 4: Actuators map commands into torque channels . . . 22

9.5 Step 5: Dynamics propagates the state . . . 22

9.6 Step 6: Logs and GUI summarize the run . . . 23

10 What a Beginner Should Notice First in Each Screenshot **23**

10.1 Top-level model . . . 23

10.2 GUI . . . 23

11 Current Validated Branch: Accuracy and Limitations **23**

11.1 What is meaningfully modeled now . . . 24

11.2 What is structurally present but currently simplified . . . 24

11.3 How to phrase the model honestly . . . 24

12 Quick Reference **25**

12.1 Acronyms . . . 25

12.2 Fast reading checklist . . . 25

A Subsystem Screenshot Gallery **25**

B Sources and Further Reading **31**

Sources and Further Reading . . . 31

1 What the Model Is

The repository contains a programmatic MATLAB/Simulink attitude determination and control system simulator for a 3U-class CubeSat mission. The simulator is organized around the canonical ADCS pipeline:

```
truth environment -> sensors -> estimation/control -> actuators -> dynamics
```

The files that matter most when you want to understand the model are shown in Table 1.

File	Why it matters
`README.md`	High-level entry point, quick start, and repository map.
`build_adcs_model.m`	Creates the top-level model, declares top-level subsystem blocks, and defines cross-subsystem signal routing.
`builders/build_environment.m`	Defines the ENVIRONMENT subsystem interface and its embedded MATLAB Function logic.
`builders/build_sensors.m`	Defines the SENSORS subsystem and how truth states are converted into measurements.
`builders/build_control.m`	Defines the CONTROL subsystem and how measurements map to commands.
`builders/build_actuators.m`	Defines the ACTUATORS subsystem and its torque-generation interfaces.
`builders/build_dynamics.m`	Defines the DYNAMICS subsystem, integrators, disturbance terms, and state feedback.
`init_adcs_params.m`	Loads base-workspace parameters, ephemeris arrays, spacecraft properties, sensor properties, actuator limits, and simulation settings.
`launch_adcs_gui.m`	Defines the current interactive dashboard and therefore the actual user-facing controls.
`simulate_adcs_case.m`	Shows the run workflow: initialize parameters, apply overrides, rebuild the model, simulate, summarize.
`summarize_adcs_simulation.m`	Defines the derived metrics and the exact meaning of the main logged outputs.
`reference.md`	Plain-language architecture, port map, and parameter quick-reference.
`skills.md`	Domain notes for the intended physics, estimation, and control models.

Table 1: Files that function as the real source of truth when reading the model.

1.1 Generated versus authored content

`adcs_sim.slx` is generated output. It is useful for visualization, but it is not the best place to answer questions such as "what does port 4 mean?" or "where does this line come from?" Those answers live in the builders and in the top-level build script.

2 How to Read the Model Without Getting Lost

The beginner-friendly way to read this project is:

1. Open `README.md` and `reference.md` first.
2. Open `build_adcs_model.m` and read the comments above the top-level routing.
3. Open the top-level Simulink model and identify the five major subsystems from left to right.
4. For any connector that is unclear, look up the source and destination port order in `build_adcs_model.m` or `reference.md`.
5. If a block inside a subsystem is unclear, open the matching `builders/build_*.m` file and read the embedded MATLAB Function code generator for that block.
6. Use `init_adcs_params.m` to understand where each constant, initial condition, and limit comes from.
7. Use `summarize_adcs_simulation.m` to understand what the logged outputs are later turned into.

In other words: use the diagram to orient yourself, and use the MATLAB source to disambiguate the diagram.

2.1 How connectors work in this project

Most of the confusion in a generated Simulink model comes from the fact that lines are created by port number, not by prose. This project is unusually readable once you know that the builder comments document the port contract explicitly.

Example:

```
ENVIRONMENT output port mapping:
  1 = Sun_vec_ECI   2 = Moon_vec_ECI   3 = B_ECI
  4 = pos_ECI       5 = vel_ECI        6 = eclipse_flag
```

Therefore this line in the source:

```
add_line(mdl, 'ENVIRONMENT/1', 'SENSORS/3', ...)
```

means:

```
Sun_vec_ECI  ->  Sun_ECI
```

Once you adopt that reading style, the model becomes much easier to follow.

3 End-to-End Simulation Workflow

The run pipeline from parameters to outputs is shown below:

```
init_adcs_params
    |
    +-- loads constants, initial conditions, ephemeris arrays, orbit data,
        sensor specs, actuator limits, and simulation settings into the base workspace
    |
build_adcs_model
    |
    +-- creates the top-level Simulink model and subsystem internals
    |
sim('adcs_sim')
    |
    +-- propagates the model and logs q_log, omega_log, h_W_log
    |
summarize_adcs_simulation
    |
    +-- derives excursion, norm error, peak/rms rates, kinetic energy,
        wheel utilization, and total angular momentum metrics
    |
launch_adcs_gui
    |
    +-- displays plots, metrics, and the effective parameter set
```

3.1 What the GUI actually does

The GUI is not an alternate simulation engine. It is a front-end around the same standard sequence:

```
init_adcs_params -> apply selected overrides -> build_adcs_model -> sim -> summarize
```

The GUI therefore changes the run by changing selected workspace values before the model is rebuilt and simulated.

4 Top-Level Architecture and Data Flow

Figure 1 shows the generated top-level Simulink diagram. Read it left to right first, then notice the feedback lines from DYNAMICS back to SENSORS and CONTROL.

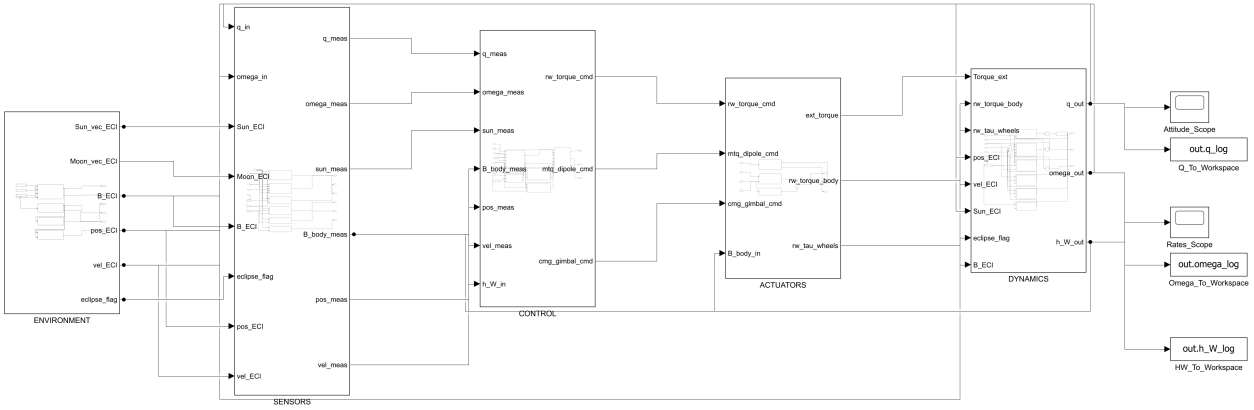


Figure 1: Top-level generated Simulink model. The structure is left-to-right, but it closes the loop through attitude and rate feedback from DYNAMICS back into SENSORS, and wheel-momentum feedback from DYNAMICS into CONTROL.

4.1 Top-level signal flow

Table 2 summarizes the main cross-subsystem lines.

From	To	Signals
ENVIRONMENT	SENSORS	`Sun_vec_ECI`, `Moon_vec_ECI`, `B_ECI`, `pos_ECI`, `vel_ECI`, `eclipse_flag`
SENSORS	CONTROL	`q_meas`, `omega_meas`, `sun_meas`, `B_body_meas`, `pos_meas`, `vel_meas`
DYNAMICS	CONTROL	`h_W_out`
CONTROL	ACTUATORS	`rw_torque_cmd`, `mtq_dipole_cmd`, `cmg_gimbal_cmd`
SENSORS	ACTUATORS	`B_body_meas` for magnetorquer torque computation
ACTUATORS	DYNAMICS	`ext_torque`, `rw_torque_body`, `rw_tau_wheels`
DYNAMICS	SENSORS	`q_out`, `omega_out` as truth feedback
DYNAMICS	Workspace	`q_log`, `omega_log`, `h_W_log` through To Workspace blocks

Table 2: Top-level dataflow across the major subsystems.

4.2 What this architecture means physically

The simulator is trying to express the following idea:

1. The spacecraft exists in an orbital environment with sunlight, Earth magnetic field, position, velocity, and eclipse state.
2. Sensors do not see the "truth" perfectly; they produce measurements derived from the truth state.
3. Control logic interprets those measurements and decides what torques or actuator commands to apply.

4. Actuators turn those commands into body torques or internal momentum changes.
5. Dynamics integrates the rotational equations of motion and updates the true attitude, body rate, and reaction-wheel momentum state.
6. Those updated truths become the next sensor inputs, and the loop continues.

5 Inputs to the Model

There are three distinct kinds of inputs:

1. **Base-workspace parameters** from `init_adcs_params.m`.
2. **Ephemeris data** from `ephemeris_2026_weekly.csv`.
3. **GUI override fields** exposed by `launch_adcs_gui.m`.

5.1 Base-workspace parameter categories

The model takes many workspace parameters, but they are easier to understand if grouped by role rather than by variable name. Table 3 shows the categories that matter most to a new reader.

Category	What it includes
Spacecraft properties	Mass, inertia tensor, 3U geometry, face normals, face areas, and center-of-pressure offsets used by disturbance models.
Initial attitude state	Initial quaternion `q0`.
Initial rotational state	Initial body rate `omega0` and initial reaction-wheel momentum `h_W0`.
Simulation settings	Fixed-step size `dt` and run length `t_end`.
Ephemeris truth arrays	Time grid, Sun vectors, and Moon vectors loaded from CSV.
Orbit parameters	Earth constants, altitude, inclination, eccentricity, RAAN, argument of perigee, true anomaly, orbital period, and mean motion.
Sensor parameters	FSS noise and field of view, star-tracker exclusion angles and rates, IMU error parameters, magnetometer bias and noise, GNSS noise and sample rates.
Actuator parameters	Reaction wheel torque, momentum, speed, inertia, geometry; magnetorquer max dipole; CMG torque and momentum limits.
Control parameters	PD gains, B-dot gain, dump gain, unload threshold, and mode thresholds.
Environmental constants	Solar flux, speed of light, Earth magnetic-dipole anchor, atmospheric density anchor, drag coefficient, SRP coefficient, residual dipole.
Estimator tuning	Noise proxies used by the intended TRIAD/MEKF path.

Table 3: High-level categories of workspace inputs loaded before simulation.

5.2 CSV ephemeris input

The file `ephemeris_2026_weekly.csv` provides timestamped Sun and Moon direction vectors. In the current validated branch, the `Ephemeris_Truth` block interpolates these arrays to produce the truth-side `Sun_vec_ECI` and `Moon_vec_ECI` signals. This means that even though several other environment blocks are presently simplified, the Sun and Moon direction path is still grounded in preloaded ephemeris data rather than a hard-coded constant.

5.3 GUI controls

Figure 2 shows the current dashboard as implemented in `launch_adcs_gui.m`.

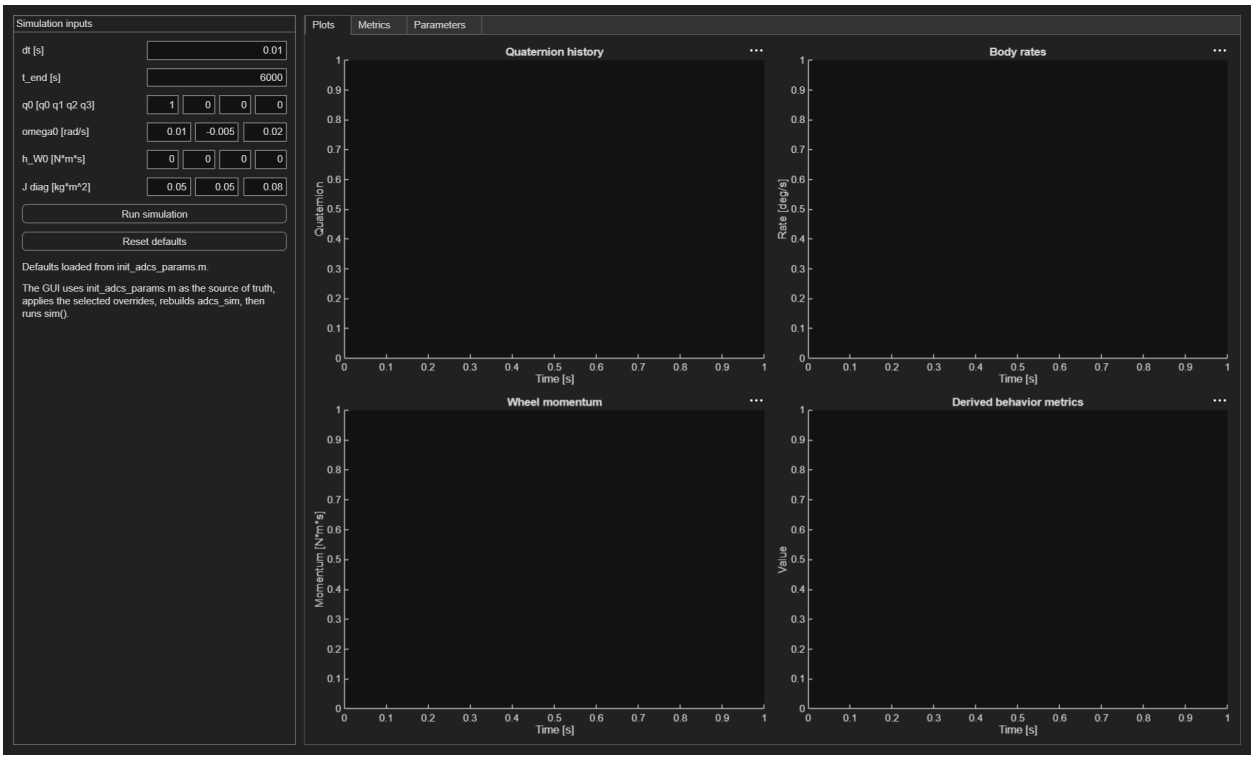


Figure 2: Current GUI before running a simulation. The source-controlled GUI uses numeric edit fields, not sliders.

Important clarification: the current GUI does *not* contain sliders anywhere in the source-controlled code. It uses numeric input fields instead. For the purpose of understanding the model, these numeric fields are the controls that determine the run.

Table 4 explains what each control changes and how that affects the spacecraft behavior represented by the model.

Control	What it changes in the model	Expected effect on behavior
`dt [s]`	Fixed solver step used by the top-level model.	Smaller values give finer time resolution and usually more accurate integration at the cost of runtime and output size. Larger values make the run cheaper but can smear or under-resolve fast rotational behavior.
`t_end [s]`	Simulation horizon.	Longer runs show more cumulative attitude evolution and make orbit-scale behavior easier to inspect. Shorter runs are better for smoke tests and debugging.
`q0 [q0 q1 q2 q3]`	Initial spacecraft attitude quaternion. The GUI normalizes it if needed before running.	Changes the starting orientation of the body frame relative to the inertial frame. This affects the initial Sun vector seen in the body frame, the starting gravitational alignment, and any initial attitude excursion relative to the reference.
`omega0 [rad/s]`	Initial body angular velocity.	Larger initial rates produce faster tumbling, larger peak rate norms, higher rotational kinetic energy, and more visible attitude change over time.
`h_W0 [N*m*s]`	Initial reaction-wheel stored momentum state.	In the architecture this matters for momentum management. In the current validated branch it mainly affects bookkeeping and GUI display because the present control and actuator paths are fallback implementations with zero commanded torques.
`J diag [kg*m^2]`	Diagonal inertia entries used to form the spacecraft inertia matrix.	Higher inertia about an axis means that the same external torque causes less angular acceleration about that axis. It also changes gravity-gradient coupling and rotational kinetic energy.

Table 4: Current GUI controls and their physical meaning.

5.4 What the GUI does not change

The GUI does not currently expose every parameter in `init_adcs_params.m`. For example, it does not expose sensor noise settings, actuator max limits, control gains, orbital elements, or

environmental disturbance constants directly. Those still come from the parameter file.

6 Outputs from the Model

The current model provides outputs at three levels:

1. **Live diagram outputs** through the two Scope blocks.
2. **Workspace logs** through the three To Workspace blocks.
3. **Derived summaries** through `summarize_adcs_simulation.m` and the GUI tables.

6.1 Primary logged outputs

Table 5 describes the main output products.

Name	Type	Meaning
`q_log`	`timeseries`	Time history of the body attitude quaternion after quaternion normalization.
`omega_log`	`timeseries`	Time history of the body angular rate vector in rad/s.
`h_W_log`	`timeseries`	Time history of the reaction-wheel momentum state.
`Attitude_Scope`	Simulink scope	Visual trace of the quaternion output inside the model.
`Rates_Scope`	Simulink scope	Visual trace of the angular-rate output inside the model.

Table 5: Primary model outputs.

6.2 Derived metrics

`summarize_adcs_simulation.m` computes a second layer of outputs that are especially useful to a beginner because they convert raw state histories into engineering summaries. These include:

- final and peak attitude excursion from the initial quaternion
- maximum quaternion norm error
- final, peak, and RMS body-rate norm
- peak rate by axis and RMS rate by axis
- peak angular acceleration
- final and peak rotational kinetic energy
- final and peak reaction-wheel momentum norm
- estimated wheel-speed utilization
- total angular momentum norm

Figure 3 shows the GUI after a run, where plots, metrics, and parameter tables are presented together.

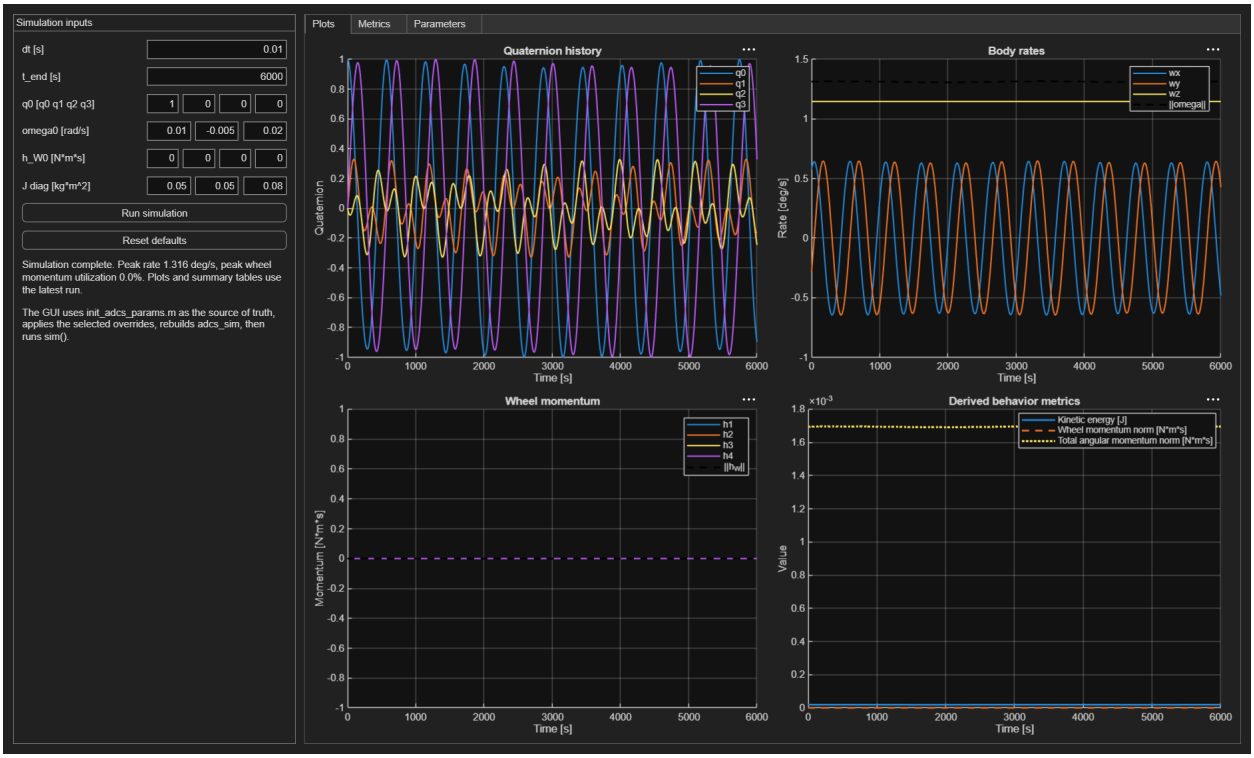


Figure 3: GUI after a simulation run. The plots tab visualizes quaternion, rate, and wheel-momentum histories, while the metrics and parameters tabs summarize the run numerically.

Figure 4 shows a compact standalone summary plot exported from a default run.

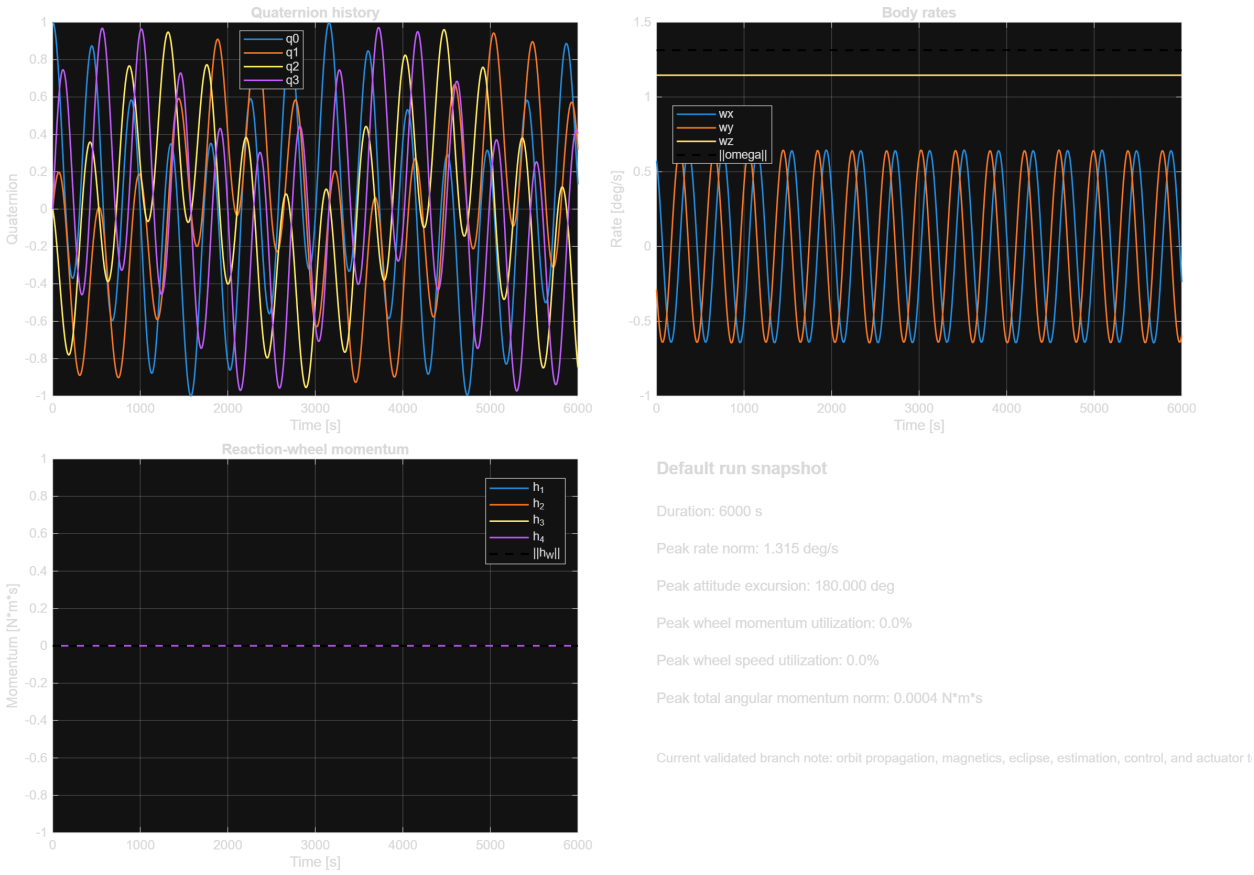


Figure 4: Default run summary exported from the repository. The current validated branch produces nontrivial attitude and rate histories, but zero wheel-momentum growth because the present actuator path is a zero-torque fallback.

6.3 Default-run interpretation

With the repository defaults, a baseline run produces approximately:

- duration: 6000 s
- peak rate norm: about 1.315 deg/s
- peak attitude excursion: about 180 deg
- peak wheel momentum utilization: 0%
- peak wheel speed utilization: 0%

Those numbers are internally consistent with the current branch: the rigid-body state evolves, but control and actuator fallbacks do not build wheel momentum or apply closed-loop stabilizing torques.

7 Subsystem-by-Subsystem Guide

7.1 ENVIRONMENT

Conceptually, ENVIRONMENT answers: "What does space look like around the spacecraft right now?"

Inputs and outputs. It has no top-level truth-state inputs; it uses the simulation clock and workspace parameters. It outputs:

- `Sun_vec_ECI`
- `Moon_vec_ECI`
- `B_ECI`
- `pos_ECI`
- `vel_ECI`
- `eclipse_flag`

What the blocks are meant to represent.

- `Ephemeris_Truth`: truth-side Sun and Moon direction in ECI, interpolated from CSV.
- `Orbit_Propagator`: orbital state generation.
- `Magnetic_Field_Model`: Earth field in ECI.
- `Eclipse_Model`: whether the spacecraft is illuminated.

What the validated branch actually does.

- `Ephemeris_Truth` is active and interpolates Sun and Moon unit vectors from the CSV arrays.
- `Orbit_Propagator` is currently a fixed-state fallback returning constant position and velocity.
- `Magnetic_Field_Model` is currently a constant-field fallback.
- `Eclipse_Model` is currently a constant-sunlit fallback.

That means the environment dataflow is correct, but the present numerical fidelity is uneven.

7.2 SENSORS

Conceptually, SENSORS answers: "Given the true spacecraft state and the environment, what would the onboard sensors report?"

Inputs.

- true attitude quaternion `q_in`
- true angular rate `omega_in`
- inertial Sun vector, Moon vector, magnetic field
- eclipse flag
- inertial position and velocity

Outputs.

- `q_meas`
- `omega_meas`
- `sun_meas`
- `B_body_meas`
- `pos_meas`
- `vel_meas`

What is physically active right now.

- `Sun_Sensor_Suite` rotates the inertial Sun vector into the body frame and zeros it when eclipse is active.
- `Magnetometer` rotates the inertial magnetic-field vector into the body frame.

What is currently simplified.

- `Star_Tracker` is a quaternion passthrough fallback.
- `IMU` is an angular-rate passthrough fallback.
- `GNSS_Position` and `GNSS_Velocity` are position and velocity passthrough fallbacks.

So, in the present branch, SENSORS is best understood as a mixed "real transform plus passthrough" layer.

7.3 CONTROL

Conceptually, CONTROL answers: "Given measurements and mission logic, what should the spacecraft command next?"

Inputs.

- measured attitude and angular rate

- measured Sun vector and magnetic field in the body frame
- measured position and velocity
- current reaction-wheel momentum state

Outputs.

- reaction-wheel torque command
- magnetorquer dipole command
- CMG gimbal command

Architectural role of each block.

- `Onboard_Ephemeris`: controller-side environmental prediction.
- `Attitude_Estimator`: estimated quaternion and rate.
- `Reference_Generator`: desired pointing and desired angular rate.
- `Mode_Manager`: detumble / coarse / fine pointing logic.
- `Control_Law`: mapping from estimated state and reference to actuator commands.

Validated-branch behavior.

- `Onboard_Ephemeris` outputs constant reference vectors.
- `Attitude_Estimator` normalizes `q_meas`, forces a positive scalar sign convention, and passes through `omega_meas`.
- `Reference_Generator` returns an identity quaternion and zero desired rate.
- `Mode_Manager` returns a constant mode identifier of 2.
- `Control_Law` outputs zero commands for all actuators.

This means CONTROL is presently carrying the *interface contract* of a flight control stack, but not yet the intended flight-quality control behavior described in the domain notes.

7.4 ACTUATORS

Conceptually, ACTUATORS answers: "How do commanded actuator actions become torques or momentum changes in the body?"

Inputs.

- reaction-wheel torque command

- magnetorquer dipole command
- CMG gimbal command
- measured body-frame magnetic field

Outputs.

- `ext_torque`
- `rw_torque_body`
- `rw_tau_wheels`

Why there are three outputs instead of one.

This is an important reading detail. The model separates:

- external body torque from MTQ and CMG,
- reaction-wheel torque applied to the spacecraft body,
- wheel-axis torques that integrate into the wheel-momentum state.

That is good modeling structure, because reaction wheels do not just apply a body torque; they also change the wheels' internal stored angular momentum.

Validated-branch behavior.

All three actuator blocks are currently zero-output fallbacks. As a result:

- `ext_torque` is zero,
- `rw_torque_body` is zero,
- `rw_tau_wheels` is zero.

This is why the default run shows zero wheel-momentum buildup and zero wheel-speed utilization.

7.5 DYNAMICS

Conceptually, DYNAMICS answers: "Given the current torques and internal momentum state, how does the spacecraft rotate next?"

Inputs.

- commanded external torque
- reaction-wheel body torque
- reaction-wheel wheel-axis torques

- position, velocity, Sun vector, eclipse flag, magnetic field

Outputs.

- `q_out`
- `omega_out`
- `h_W_out`

Physically active blocks in the validated branch.

- `Euler_RHS`
- `Gravity_Gradient`
- `Quat_RHS`
- `Quat_Norm`
- the `Integ_omega`, `Integ_hW`, and `Integ_q` integrators

Currently simplified blocks.

- `Aero_Drag_Torque`
- `SRP_Torque`
- `Residual_Mag_Torque`

Why DYNAMICS is the heart of the current branch.

Even with the simplified control and actuator layers, DYNAMICS still integrates a genuine rigid-body rotational system with quaternion kinematics and gravity-gradient torque. That is why the default run still produces a changing quaternion and changing angular-rate history.

8 Physics and Math Background for Hard-to-Read Parts

This section is intentionally short and practical. It is not a full ADCS textbook; it is the minimum background a beginner needs to decode the model.

8.1 Frames

The repository primarily uses:

- **ECI** for inertial quantities such as Sun direction, magnetic field, position, and velocity.
- **Body frame** for spacecraft-fixed quantities such as body rates, body torque, sensor boresights, and actuator axes.

Whenever you see a block that rotates a vector from ECI to body, it is answering the question: "What does this inertial direction look like from the spacecraft's own point of view?"

8.2 Quaternions

The quaternion convention is scalar-first:

$$q = [q_0,\ q_1,\ q_2,\ q_3]^T$$

with unit norm:

$$\|q\| = 1$$

The quaternion derivative block uses:

$$\dot{q} = \frac{1}{2}\Omega(\omega)q$$

where the matrix in the code corresponds to the scalar-first convention. The model then renormalizes the quaternion after integration so numerical drift does not slowly violate the unit-norm constraint.

8.3 Rigid-body rotational dynamics

The core rotational equation implemented in `Euler_RHS` is:

$$\dot{\omega} = J^{-1}\left(\tau_{\text{ext}} - \omega \times H_{\text{total}} - \tau_{\text{rw,body}}\right)$$

with

$$H_{\text{total}} = J\omega + A_W h_W$$

This is a strong clue that the model is trying to keep track of both:

- the spacecraft body's angular momentum, and
- the stored angular momentum in the reaction wheels.

With the current default parameter set, the inertia tensor is the uniform-density baseline implied by the declared 4 kg, $0.10 \times 0.10 \times 0.30$ m bus geometry:

$$J = \text{diag}([0.0333,\ 0.0333,\ 0.0067])\ \text{kg}\cdot\text{m}^2$$

8.4 Gravity-gradient torque

The gravity-gradient block computes the classic body-frame disturbance torque:

$$\tau_{gg} = \frac{3\mu}{R^3}\left(r_{\text{body}} \times J r_{\text{body}}\right)$$

Physically, this is the torque that appears because different parts of an extended spacecraft feel slightly different gravitational pull when the body is not inertially isotropic. In the current validated branch, the quaternion-to-DCM mapping used here matches the same ECI-to-body convention used by the active Sun-sensor and magnetometer transforms.

8.5 Magnetorquer torque

The intended magnetorquer relation is:

$$\tau = m \times B$$

where m is the commanded dipole moment and B is the local magnetic field in the body frame. The current validated branch preserves this interface, even though the present actuator block is a fallback.

8.6 Reaction-wheel momentum

Reaction wheels are internal actuators. They do not create net system angular momentum out of nothing; they exchange angular momentum between the wheel rotors and the spacecraft body. That is why the model tracks wheel momentum h_W separately and feeds it both into DYNAMICS and CONTROL.

9 Detailed End-to-End Data Path

This section answers the user-level question: "How does data actually move from environment to workspace?"

9.1 Step 1: Environment truth is assembled

At each simulation time, ENVIRONMENT produces:

- Sun direction in ECI from interpolated ephemeris
- Moon direction in ECI from interpolated ephemeris
- spacecraft position and velocity
- Earth magnetic field in ECI
- eclipse status

9.2 Step 2: Sensors reinterpret truth as measurements

SENSORS consumes truth attitude and rate from DYNAMICS and truth environment signals from ENVIRONMENT. It then emits:

- measured quaternion
- measured rate
- body-frame Sun vector
- body-frame magnetic field
- measured position and velocity

In the current branch, the Sun and magnetometer transformations are active, while several other sensors are passthrough fallbacks.

9.3 Step 3: Control decides what to command

CONTROL takes those measurements and the wheel-momentum state, then forms:

- desired reference attitude and rate
- current mode
- actuator commands

In the present branch this stage is structurally present but numerically simplified: the reference is fixed, mode is fixed, and commanded torques are zero.

9.4 Step 4: Actuators map commands into torque channels

ACTUATORS separates the results into:

- body torque from non-wheel actuators
- body torque from wheel reaction
- wheel-axis torques for wheel momentum integration

This separation is conceptually important even though the current fallback outputs are zero.

9.5 Step 5: Dynamics propagates the state

DYNAMICS sums commanded and disturbance torques, computes angular acceleration, integrates angular rate and quaternion, integrates wheel momentum, and emits:

- normalized attitude quaternion
- body angular rate
- wheel momentum state

9.6 Step 6: Logs and GUI summarize the run

The model sends:

- `q_out` to `Q_To_Workspace` as `q_log`
- `omega_out` to `Omega_To_Workspace` as `omega_log`
- `h_W_out` to `HW_To_Workspace` as `h_W_log`

Then `summarize_adcs_simulation.m` turns those logs into the metrics shown in the GUI.

10 What a Beginner Should Notice First in Each Screenshot

10.1 Top-level model

When you open the top-level model, do not try to follow every line at once. First answer only four questions:

1. What are the five major subsystems?
2. Which signals move left to right?
3. Which signals feed back from DYNAMICS?
4. Which outputs are logged to the workspace?

Once you can answer those, the whole model stops looking like a random wire jungle.

10.2 GUI

When you open the GUI, look for:

1. the left column of editable run inputs
2. the run and reset buttons
3. the status text telling you what the GUI will do
4. the tabs on the right: plots, metrics, parameters

That is the whole interaction model.

11 Current Validated Branch: Accuracy and Limitations

It is important not to overstate what the current branch is doing.

11.1 What is meaningfully modeled now

- quaternion kinematics
- rigid-body rotational dynamics
- gravity-gradient torque
- transformation of inertial Sun and magnetic vectors into the body frame
- logging and derived-metric computation

11.2 What is structurally present but currently simplified

- orbit propagation
- Earth magnetic-field modeling
- eclipse logic
- star tracker, IMU, and GNSS noise and sample logic
- onboard ephemeris
- attitude estimation beyond simple normalization
- reference generation
- mode logic
- control torque generation
- reaction-wheel, magnetorquer, and CMG torque realization
- aerodynamic drag torque
- solar radiation pressure torque
- residual magnetic dipole torque

11.3 How to phrase the model honestly

The fairest one-sentence description of the repository today is:

> The project already contains a well-structured closed-loop ADCS Simulink architecture with physically meaningful attitude-state propagation, but several environment, estimation, control, and actuator blocks are still running compile-safe fallback implementations in the validated branch.

That statement is much more accurate than simply calling the current branch a fully high-fidelity orbital behavior simulator.

12 Quick Reference

12.1 Acronyms

Acronym	Meaning
ADCS	Attitude Determination and Control System
ECI	Earth-Centered Inertial frame
FSS	Fine Sun Sensor
GNSS	Global Navigation Satellite System receiver
GUI	Graphical User Interface
IMU	Inertial Measurement Unit
MEKF	Multiplicative Extended Kalman Filter
MTQ	Magnetorquer
PD	Proportional-Derivative
RW	Reaction Wheel
SRP	Solar Radiation Pressure
TRIAD	Two-vector deterministic attitude determination method

Table 6: Acronyms used throughout the model and this guide.

12.2 Fast reading checklist

If you only have ten minutes with the project, inspect these in order:

1. `README.md`
2. `reference.md`
3. `build_adcs_model.m`
4. top-level Simulink diagram
5. `init_adcs_params.m`
6. `launch_adcs_gui.m`
7. `builders/build_dynamics.m`

A Subsystem Screenshot Gallery

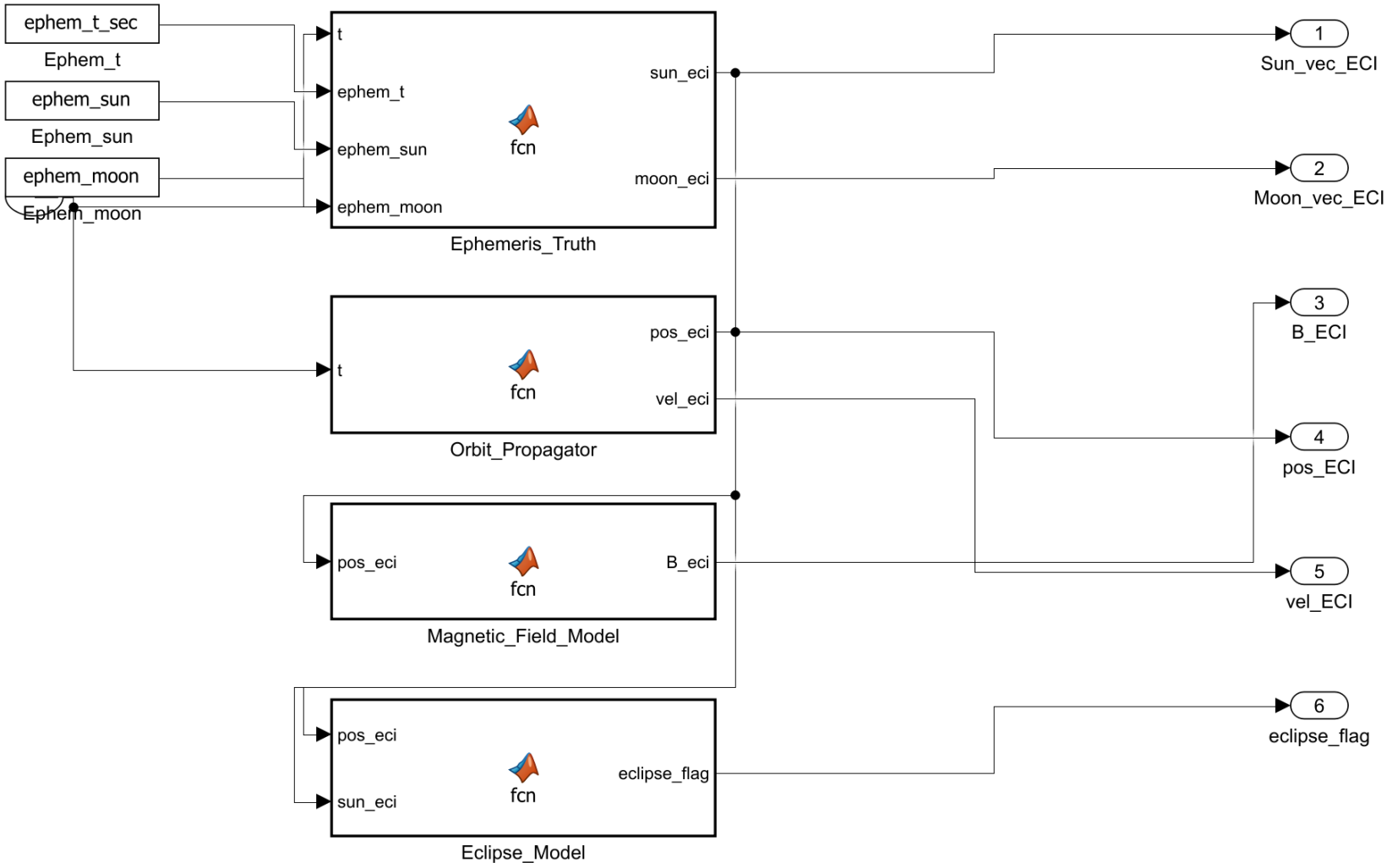


Figure 5: ENVIRONMENT subsystem. Start here when you want to understand where the truth-side Sun, Moon, orbit, magnetic field, and eclipse signals come from.

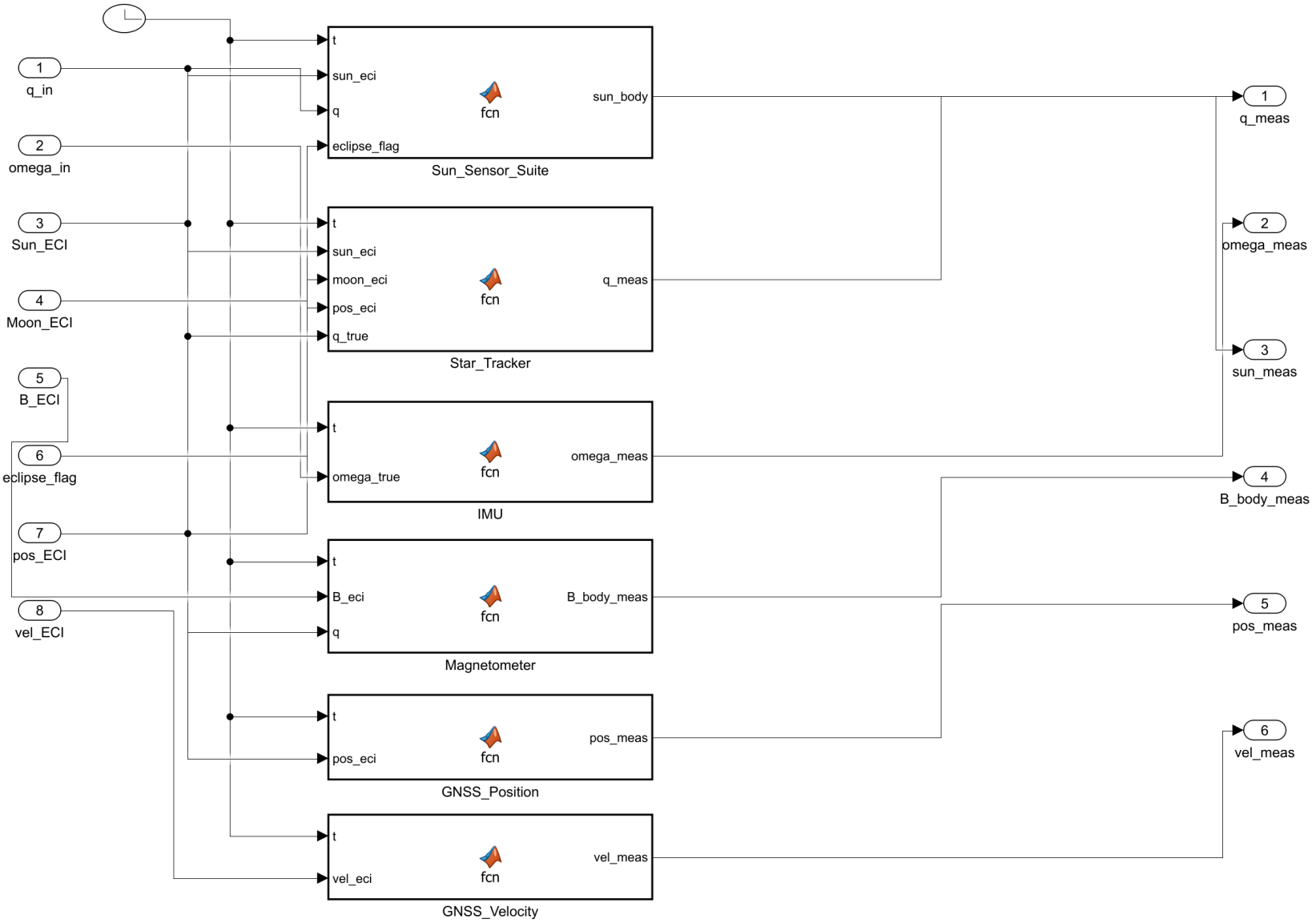


Figure 6: SENSORS subsystem. This is where true state and environment are turned into measurements.

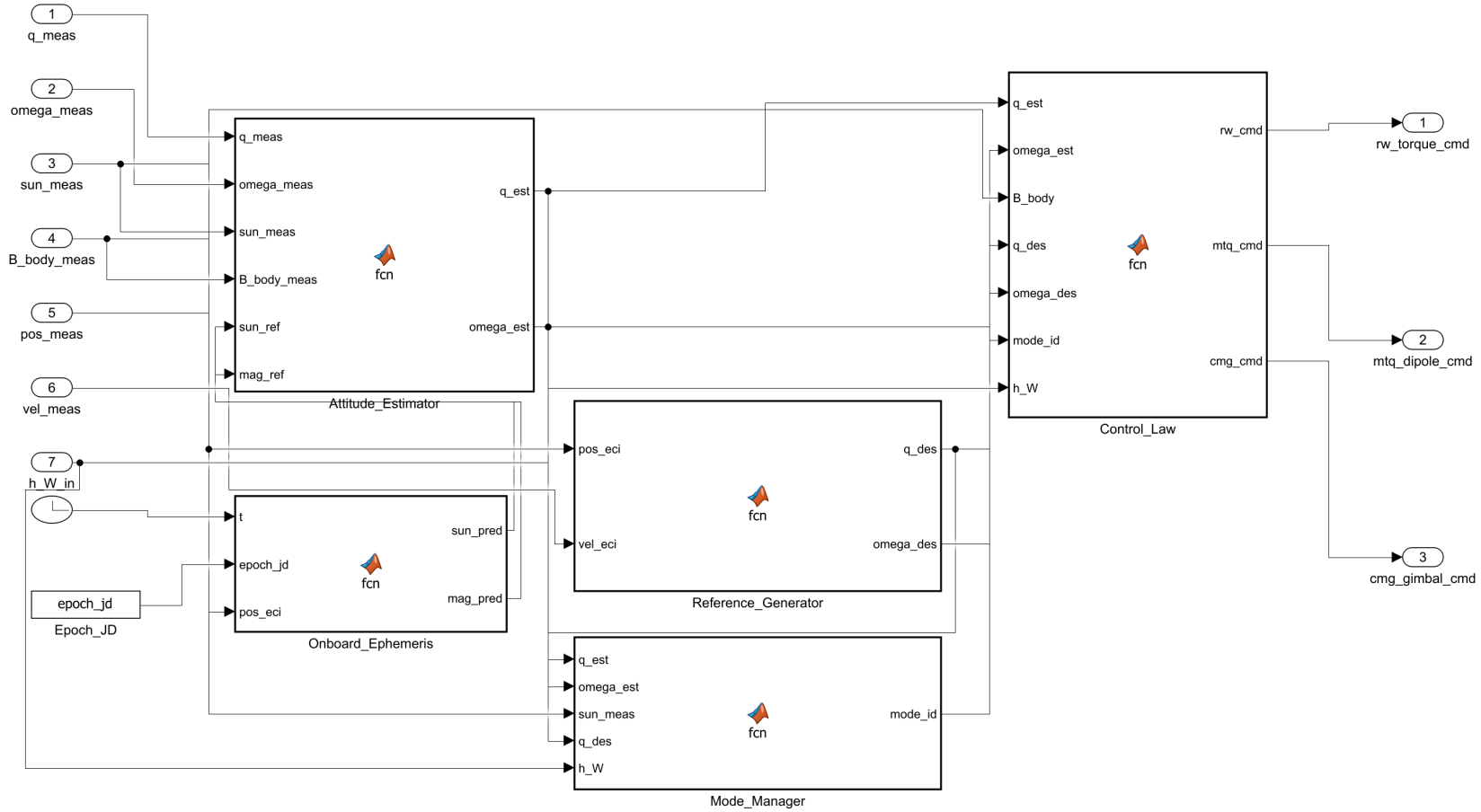


Figure 7: CONTROL subsystem. The current branch preserves the full interface structure even though several blocks are simplified fallbacks.

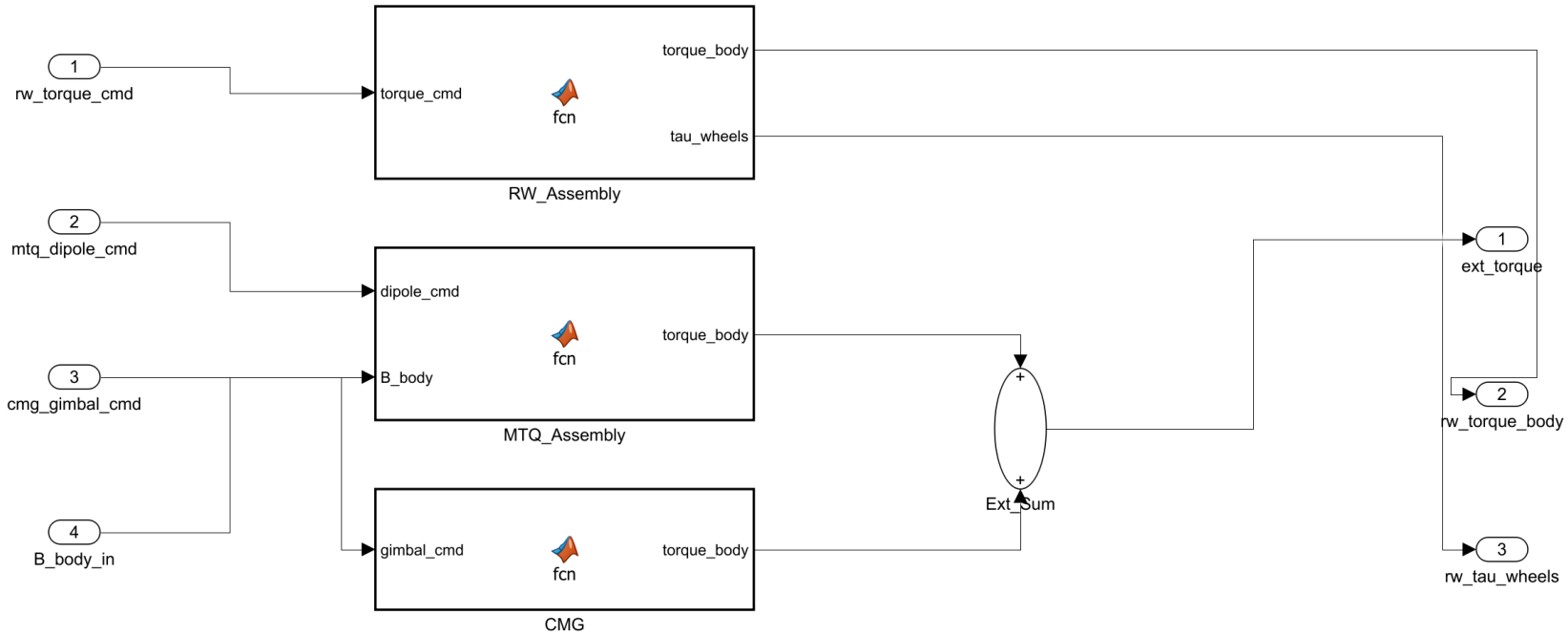


Figure 8: ACTUATORS subsystem. Note the intentional split between external torque, wheel-body torque, and wheel-axis torque.

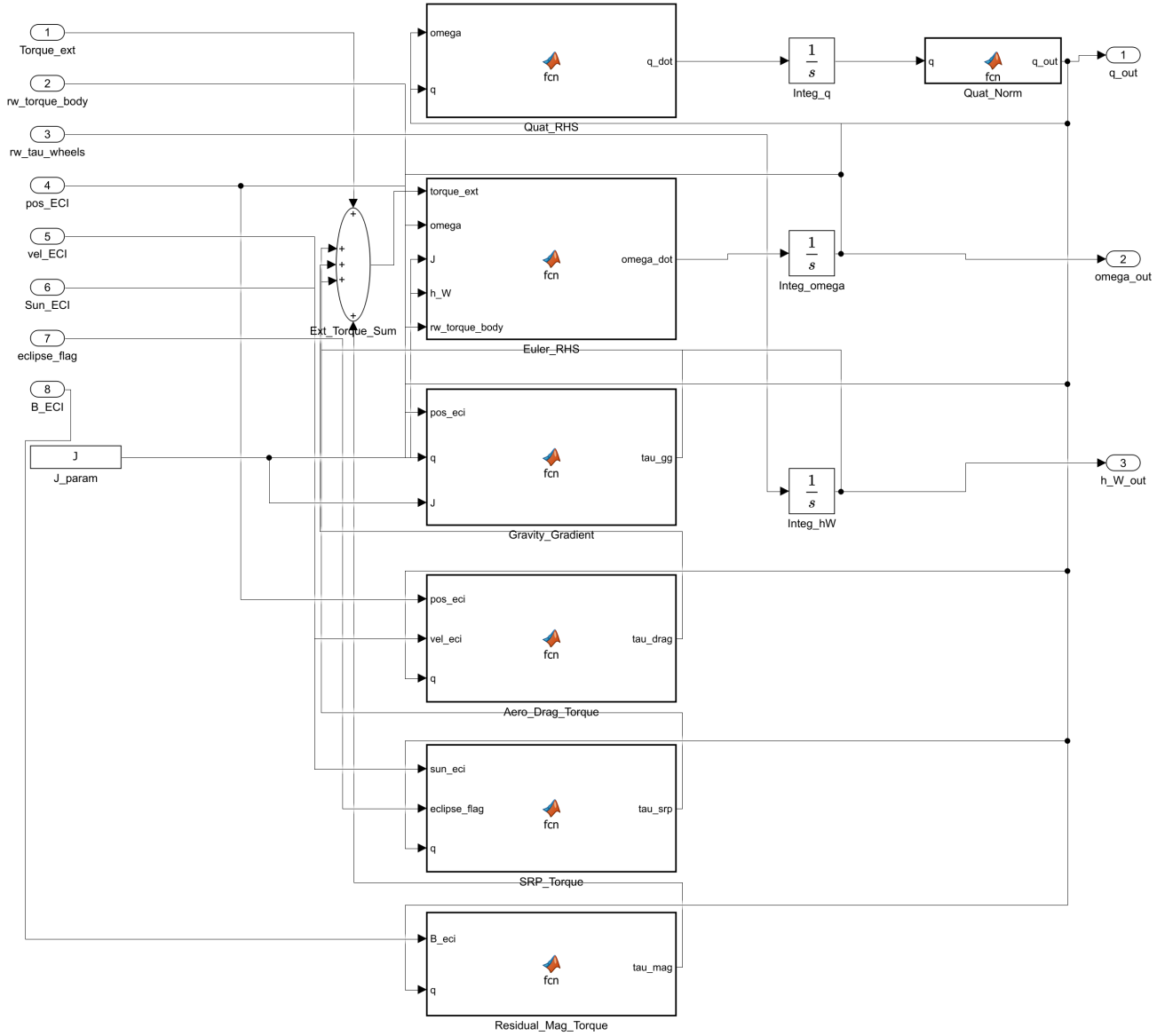


Figure 9: DYNAMICS subsystem. This is the core of the current validated branch and the main place to study quaternion propagation, torque summation, and rigid-body state integration.

B Sources and Further Reading

Repository sources

- `README.md`
- `reference.md`
- `init_adcs_params.m`
- `build_adcs_model.m`
- `builders/build_environment.m`
- `builders/build_sensors.m`
- `builders/build_control.m`
- `builders/build_actuators.m`
- `builders/build_dynamics.m`
- `launch_adcs_gui.m`
- `simulate_adcs_case.m`
- `summarize_adcs_simulation.m`
- `skills.md`

External ADCS and modeling references

- F. Landis Markley and John L. Crassidis, *Fundamentals of Spacecraft Attitude Determination and Control*. Springer, 2014.
- Hanspeter Schaub and John L. Junkins, *Analytical Mechanics of Space Systems*. AIAA.
- Howard D. Curtis, *Orbital Mechanics for Engineering Students*. Elsevier.
- Marcel J. Sidi, *Spacecraft Dynamics and Control*. Cambridge University Press.
- Oliver Montenbruck and Eberhard Gill, *Satellite Orbits*. Springer.
- MathWorks documentation for Simulink subsystem wiring, Scope blocks, and To Workspace logging: https://www.mathworks.com/help/simulink/
- NASA Small Spacecraft Systems Virtual Institute overview materials for GNC and spacecraft subsystem context: https://www.nasa.gov/smallsat-institute/
- VectorNav math and inertial-navigation primers for quaternion and IMU background: https://www.vectornav.com/resources/inertial-navigation-primer